

W11D04 — Separation of Concerns in Practice

ML Engineer Journey | Phase 3 — Docker & Clean Architecture | 21 Mei 2026

1. Apa Itu Separation of Concerns

Separation of Concerns (SoC) adalah prinsip bahwa setiap modul, class, atau fungsi harus punya **satu alasan untuk berubah**. Kalau router berubah karena logika bisnis berubah — SoC rusak. Kalau service berubah karena format HTTP response berubah — sama saja.

Analogi Rumah Sakit: Resepsionis menerima dan memverifikasi, dokter mendiagnosis, apoteker mengambil obat. Kalau resepsionis mulai mendiagnosis, atau apoteker mengubah dosis — sistem itu rusak. Kode dengan SoC yang buruk adalah 'dokter yang sekaligus jaga meja resepsionis.'

2. Empat Layer dan Tanggung Jawabnya

Layer / File	Boleh	DILARANG
Router routers/predict.py	Terima request, validasi format (Pydantic), delegate ke service, return response	Business logic, exception translation, HTTP knowledge
Service services/model_service.py	Business logic, orchestrasi, keputusan (tier, label, score)	raise ValueError / RuntimeError / Request — tau soal HTTP
Repository repositories/prediction_repo.py	Akses data, query DB/cache/file, return objek bersih atau None	Business logic, keputusan apapun, raise HTTPException
Infrastructure model/loader.py · config.py	Load model dari disk, koneksi DB, env config, detail teknis	Business logic, HTTP knowledge, exception translation

3. Aturan Emas — Lima Pertanyaan Diagnostik

Tanyakan ini ke setiap file sebelum commit. Jika ada jawaban 'ya' di tempat yang salah, pindahkan kodenya.

- File ini import FastAPI / HTTPException / Request? → Jika iya dan bukan router/main.py: **pelanggaran**.
- Ada model.predict() di file ini? → Seharusnya hanya ada di **service layer**.
- Ada raise HTTPException di service? → **Pelanggaran**. Ganti dengan raise ValueError.
- Ada business decision (if score > 0.7) di repository? → **Pelanggaran**. Pindah ke service.
- Ada database query di router? → **Pelanggaran**. Pindah ke repository.

4. None adalah Fakta, 404 adalah Interpretasi

Repository.get_by_id() mengembalikan None jika data tidak ditemukan — bukan raise HTTPException(404). Ini bukan kelalaian, ini desain yang benar.

Repository tidak tahu konteks aplikasinya. Ia tidak tahu apakah dijalankan di FastAPI, CLI, atau background job. None adalah fakta data: 'record itu tidak ada.' Keputusan apakah itu harus jadi error 404, fallback ke default, atau trigger alert — adalah urusan layer di atasnya.

Untuk aplikasi sederhana: router yang menangkap None dan raise HTTPException(404).

Untuk aplikasi kompleks: service yang menangkap None dan raise custom exception (misal: RecordNotFoundError), lalu router yang men-translate ke HTTPException — tanpa service tahu soal HTTP.

5. ValueError vs RuntimeError — dan Kenapa detail= Berbeda

Ini bukan sekadar pilihan teknis — ini keputusan keamanan dan UX sekaligus.

Aspek	ValueError (422)	RuntimeError (500)
Penyebab	Kesalahan di sisi pengguna (input tidak valid, aturan bisnis yang dilanggar)	Kesalahan di sisi server (model crash, inference gagal)
Siapa yang salah	Pengguna	Server / kode kita
User bisa apa?	Perbaiki input dan coba lagi	Tidak bisa berbuat apa-apa
detail= ke client	str(e) — informatif, bantu user memperbaiki input	Pesan generik — jangan expose traceback / path file
Kenapa tidak expose RuntimeError?		Traceback bisa berisi path file, nama library, baris kode — informasi sensitif

6. Kenapa Factory Function Milik Router, Bukan Service

get_prediction_repo() dan get_model_service() adalah dependency factory — fungsi yang membuat instance dependency dan menyuntikkannya via FastAPI Depends(). Kedua fungsi ini ada di routers/predict.py, bukan di services/model_service.py.

Alasannya satu: Depends() adalah fitur FastAPI. Kalau factory itu dipindah ke model_service.py, maka service layer harus import dari fastapi — dan service itu sekarang terikat ke FastAPI. Besok kalau service dipanggil dari CLI atau test tanpa server, import itu akan gagal atau setidaknya membawa dependency yang tidak perlu.

Prinsipnya: factory yang menggunakan mekanisme framework (Depends) harus hidup di layer yang memang milik framework — yaitu router.

7. Before vs After Refactor

Aspek	Sebelum Refactor	Setelah Refactor
Business logic	Di router atau main.py	Di services/model_service.py
Model loading	Di dalam endpoint langsung	Di model/loader.py via lifespan
Exception handling	Campur aduk, tidak konsisten	Router translate, service raise ValueError/RuntimeError
Data persistence	Tidak ada / ad-hoc	repositories/prediction_repo.py
Testability	Sulit — banyak coupling antar komponen	Tiap layer bisa di-test terpisah tanpa server
Alasan berubah per file	Banyak, tidak jelas	Satu per layer — SoC terpenuhi
Dependency ke FastAPI	Menyebar ke semua file	Hanya di router dan main.py

8. Pola Kode Kunci — Referensi Cepat

Service — raise ValueError, tidak ada FastAPI

```
class ModelService:
    def __init__(self, model: Any, repo: PredictionRepository):
        self.model = model    # diterima dari luar via DI
        self.repo = repo

    def predict(self, features: list[float]) -> dict:
        if len(features) != 5:
            raise ValueError('Butuh 5 features.') # bukan HTTPException
        try:
            arr = np.array(features).reshape(1, -1)
            prob = float(self.model.predict_proba(arr)[0].max())
        except Exception as e:
            raise RuntimeError('Inference gagal.') from e # 'from e' penting
        tier = self._determine_tier(prob)
        return self.repo.save({'prediction': ..., 'risk_tier': tier})
```

Router — translate exception, tidak ada business logic

```
@router.post('/')
async def predict(payload: PredictionRequest,
                  service: ModelService = Depends(get_model_service)):
    try:
        return service.predict(payload.features)
    except ValueError as e:
        raise HTTPException(422, detail=str(e))    # user error: expose
    except RuntimeError:
        raise HTTPException(500, detail='Inference gagal.') # server error: jangan expose
```

Repository — return None, tidak ada keputusan bisnis

```
def get_by_id(self, prediction_id: int) -> Optional[dict]:
    for record in self._store:
        if record['id'] == prediction_id:
            return record
    return None    # bukan raise HTTPException(404)
    # Keputusan apakah ini 404 atau fallback – urusan layer di atas
```

9. Poin Penting yang Harus Diingat

- SoC = setiap file punya satu alasan untuk berubah. Bukan sekedar 'pisah file.'
- raise RuntimeError(...) from e — 'from e' menyertakan original exception sebagai __cause__. Tanpa itu, traceback asli hilang.
- None adalah data, 404 adalah protokol HTTP. Repository hanya tahu data.
- Factory Depends() hidup di router — bukan di service — karena Depends() adalah fitur FastAPI.
- detail=str(e) untuk ValueError (user error, informatif), bukan untuk RuntimeError (server error, security risk).

- Struktur folder bukan dekorasi — ia adalah dokumentasi tanggung jawab tiap layer yang bisa dibaca langsung dari nama file.
- Service yang tidak import FastAPI = service yang bisa dipanggil dari mana saja: CLI, test, gRPC, scheduler.